

Mitali Bhatia

D20C 04

Blockchain

Experiment No 02

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

1. Importing Required Libraries

- **datetime** → to timestamp each block
- **hashlib** → to generate SHA-256 hashes
- **json** → to encode blocks as JSON for hashing

- **Flask** → to run the blockchain API server

2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
- store the chain
- implement Proof of Work
- validate chain integrity

3. Flask Web API

Provides endpoints:

- `/mine_block` → mine a new block
- `/get_chain` → fetch full blockchain
- `/is_valid` → verify blockchain

CODE:

```
import hashlib

import json

from datetime import datetime

from flask import Flask, jsonify

# BLOCKCHAIN CLASS

class MyBlockchain:

    def __init__(self):

        self.blocks = []

        # Create the first block (Genesis Block)

        self.add_block(1, "0")

        # Create a new block

        def add_block(self, proof,
previous_hash):

        block = {

            "block_no": len(self.blocks) + 1,

            "timestamp": str(datetime.now()),

            "proof": proof,

            "previous_hash": previous_hash

        }

        self.blocks.append(block)

        return block

    # Get the latest block

    def latest_block(self):
```

```

return self.blocks[-1]

# Proof of Work

def find_proof(self, previous_proof):

    proof = 1

    while True:

        value = proof ** 2 - previous_proof **
2
        hash_value = hashlib.sha256(

            str(value).encode()

        ).hexdigest()

        # Golden nonce condition

        if hash_value[:3] == "000":

            return proof

        proof += 1

# Generate SHA-256 hash of a block

def generate_hash(self, block):

    encoded_block = json.dumps(

        block,

        sort_keys=True

    ).encode()

    return

    hashlib.sha256(encoded_block).hexdigest()

# Check whether blockchain is valid

```

```

def verify_chain(self):

    for i in range(1, len(self.blocks)):

        current_block = self.blocks[i]

        previous_block = self.blocks[i - 1]

        # Check previous block hash

        calculated_hash =

        self.generate_hash(

            previous_block

        )

        if current_block["previous_hash"] !=

        calculated_hash:

            return False

        # Check Proof of Work

        previous_proof =

        previous_block["proof"]

        current_proof =

        current_block["proof"]

        value = current_proof ** 2 -

        previous_proof ** 2

        hash_value = hashlib.sha256(

            str(value).encode()

        ).hexdigest()

```

```

        if hash_value[:3] != "000":

            return False

        return True

# FLASK APPLICATION

app = Flask(__name__)

blockchain = MyBlockchain()

# HOME PAGE

@app.route("/", methods=["GET"])
def home():

    return """

    <h2>Python Blockchain</h2>

    <p>Blockchain server is running.</p>

    <h3>Available Operations:</h3>

    <ul>

        <li><a href="/mine">Mine One
Block</a></li>

        <li><a href="/mine_multiple/5">Mine 5
Blocks</a></li>

        <li><a href="/chain">Display
Blockchain</a></li>

        <li><a href="/validate">Validate
Blockchain</a></li>

    </ul>

    """

```

```

# MINE ONE BLOCK

@app.route("/mine", methods=["GET"])

def mine_block():

    previous_block =
blockchain.latest_block()

    previous_proof = previous_block["proof"]

    # Find the golden nonce

    new_proof = blockchain.find_proof(

        previous_proof

    )

    # Get hash of previous block

    previous_hash =
blockchain.generate_hash(

        previous_block

    )

    # Create the new block

    new_block = blockchain.add_block(

        new_proof,

        previous_hash

    )

```

```

response = {
    "message": "Block mined successfully",
    "block_number":
new_block["block_no"],

    "timestamp": new_block["timestamp"],
    "proof": new_block["proof"],
    "previous_hash":
new_block["previous_hash"]
}

```

```

return jsonify(response), 200

```

```

# MINE MULTIPLE BLOCKS

```

```

@app.route("/mine_multiple/<int:number>",
methods=["GET"])

```

```

def mine_multiple(number):

```

```

    if number <= 0:

```

```

        return jsonify({
            "error": "Number of blocks must be
greater than zero"
        }), 400

```

```

    mined_blocks = []

```

```

    for i in range(number):

```

```

        previous_block =
blockchain.latest_block()

```

```

        previous_proof =
previous_block["proof"]

```

```

        # Find proof for the new block

```

```

        new_proof = blockchain.find_proof(
            previous_proof
        )

```

```

        # Hash of previous block

```

```

        previous_hash =
blockchain.generate_hash(
            previous_block
        )

```

```

        # Add new block

```

```

        new_block = blockchain.add_block(
            new_proof,
            previous_hash
        )

```

```

        mined_blocks.append(new_block)

```

```

return jsonify({

    "message": str(number) + " blocks
mined successfully",

    "total_blocks": len(blockchain.blocks),

    "blocks_added": mined_blocks

}), 200

# DISPLAY COMPLETE BLOCKCHAIN

@app.route("/chain", methods=["GET"])

def display_chain():

    response = {

        "total_blocks": len(blockchain.blocks),

        "blockchain": blockchain.blocks

    }

    return jsonify(response), 200

# CHECK BLOCKCHAIN VALIDITY

@app.route("/validate", methods=["GET"])

def validate_chain():

    result = blockchain.verify_chain()

    if result:

        message = "Blockchain is valid."

    else:

```

```

        message = "Blockchain is not valid."

    return jsonify({

        "valid": result,

        "message": message

    }), 200

# CREATE AN INVALID BLOCK

@app.route("/make_invalid",
methods=["GET"])

def make_invalid():

    if len(blockchain.blocks) < 2:

        return jsonify({

            "message": "Mine at least one block
first."

        }), 400

# Select the latest block

invalid_block = blockchain.blocks[-1]

# Change the proof to an incorrect value

invalid_block["proof"] = 999999

return jsonify({

    "message": "An invalid block has been

```

```

created.",
    "block": invalid_block
}), 200

# START FLASK SERVER

if __name__ == "__main__":
    app.run(
        host="127.0.0.1",
        port=5000,
        debug=False,
        use_reloader=False
    )

```

OUTPUT:

Flask successfully installed:

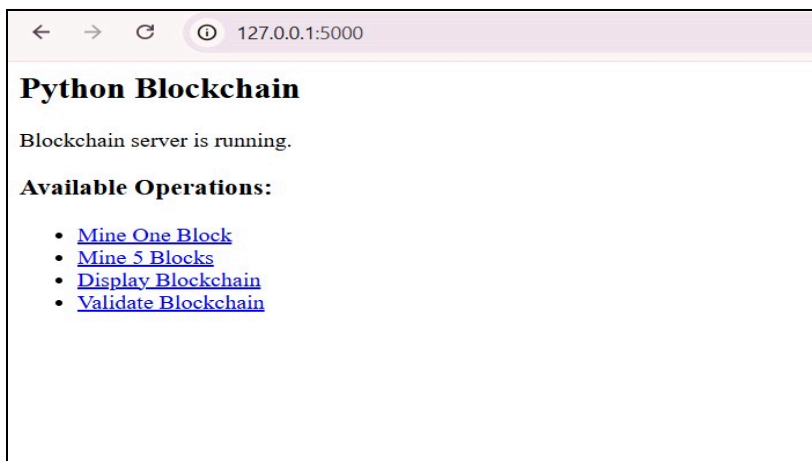
```

PS C:\Clg\Blockchain codes\Exp 02> python --version
Python 3.13.7
PS C:\Clg\Blockchain codes\Exp 02> pip install flask
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: flask in c:\users\mital\appdata\roaming\python\python313\site-packages (3.1.1)
Requirement already satisfied: blinker>=1.9.0 in c:\users\mital\appdata\roaming\python\python313\site-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in c:\users\mital\appdata\roaming\python\python313\site-packages (from flask) (8.2.1)
Requirement already satisfied: itsdangerous>=2.2.0 in c:\users\mital\appdata\roaming\python\python313\site-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in c:\users\mital\appdata\roaming\python\python313\site-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in c:\users\mital\appdata\roaming\python\python313\site-packages (from flask) (3.0.2)
Requirement already satisfied: werkzeug>=3.1.0 in c:\users\mital\appdata\roaming\python\python313\site-packages (from flask) (3.1.3)
Requirement already satisfied: colorama in c:\users\mital\appdata\roaming\python\python313\site-packages (from click>=8.1.3->flask) (0.4.6)

[notice] A new release of pip is available: 25.3 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

```

Home Page :



Mine one block :

```
← → ↻ ⓘ 127.0.0.1:5000/mine
Pretty print ☒
{
  "block_number": 2,
  "message": "Block mined successfully",
  "previous_hash": "a61e5af46fc20b681b5a6bc1ce879dfdbdd8e8a0bdc0ffa10b5b7a185ba9cdba",
  "proof": 533,
  "timestamp": "2026-09-08 21:30:12.299045"
}
```

Mine 5 blocks :

```
← → ↻ ⓘ 127.0.0.1:5000/mine_multiple/5
Pretty print ☒
{
  "blocks_added": [
    {
      "block_no": 3,
      "previous_hash": "9d972e4a2d2b0c19c45a175b826b06b1b4239d82877f81bc9a87047ae418de9a",
      "proof": 11028,
      "timestamp": "2026-09-08 21:31:03.978616"
    },
    {
      "block_no": 4,
      "previous_hash": "85e0782adf8852f40b4c877ffb496e4104a77a31ab41f907d452eaca3d4fd5a0",
      "proof": 396,
      "timestamp": "2026-09-08 21:31:03.980063"
    },
    {
      "block_no": 5,
      "previous_hash": "6ded245c43a4dbc3daa26dce3901affb8c36f489d2ed0d3f2fc454be5107cf55",
      "proof": 1430,
      "timestamp": "2026-09-08 21:31:03.984494"
    },
    {
      "block_no": 6,
      "previous_hash": "3eb9e4371877f90af8109ae4614cd3a9062fbc4bb11d3f8ac20b9db2f12a94f0",
      "proof": 1437,
      "timestamp": "2026-09-08 21:31:03.989879"
    },
    {
      "block_no": 7,
      "previous_hash": "68e97067c314a889125b05330ef706ccd589f9b0dc6f737368fd1f051d51ccd8",
      "proof": 551,
      "timestamp": "2026-09-08 21:31:03.991714"
    }
  ],
  "message": "5 blocks mined successfully",
  "total_blocks": 7
}
```

Complete Blockchain:

```
127.0.0.1:5000/chain

Pretty print ☒

{
  "blockchain": [
    {
      "block_no": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-08 21:28:29.794415"
    },
    {
      "block_no": 2,
      "previous_hash": "a61e5af46fc20b681b5a6bc1ce879dfd8dd8e8a0bdc0ffa10b5b7a185ba9cdba",
      "proof": 533,
      "timestamp": "2026-09-08 21:30:12.299045"
    },
    {
      "block_no": 3,
      "previous_hash": "9d972e4a2d2b0c19c45a175b826b06b1b4239d82877f81bc9a87047ae418de9a",
      "proof": 11028,
      "timestamp": "2026-09-08 21:31:03.978616"
    },
    {
      "block_no": 4,
      "previous_hash": "85e0782adf8852f40b4c877ffb496e4104a77a31ab41f907d452eaca3d4fd5a0",
      "proof": 396,
      "timestamp": "2026-09-08 21:31:03.980063"
    },
    {
      "block_no": 5,
      "previous_hash": "6ded245c43a4dbc3daa26dce3901affb8c36f489d2ed0d3f2fc454be5107cf55",
      "proof": 1430,
      "timestamp": "2026-09-08 21:31:03.984494"
    },
    {
      "block_no": 6,
      "previous_hash": "3eb9e4371877f90af8109ae4614cd3a9062fbc4bb11d3f8ac20b9db2f12a94f0",
      "proof": 1437,
      "timestamp": "2026-09-08 21:31:03.989879"
    },
    {
      "block_no": 7,
      "previous_hash": "68e97067c314a889125b05330ef706ccd589f9b0dc6f737368fd1f051d51ccd8",
      "proof": 551,
      "timestamp": "2026-09-08 21:31:03.991714"
    }
  ],
  "total_blocks": 7
}
```

Valid Blockchain:

```
127.0.0.1:5000/validate

Pretty print ☒

{
  "message": "Blockchain is valid.",
  "valid": true
}
```

Invalid Block:



```
127.0.0.1:5000/make_invalid
Pretty print
{
  "block": {
    "block_no": 7,
    "previous_hash": "68e97067c314a889125b05330ef706ccd589f9b0dc6f737368fd1f051d51ccd8",
    "proof": 999999,
    "timestamp": "2026-09-08 21:31:03.991714"
  },
  "message": "An invalid block has been created."
}
```

Invalid Blockchain Validation:



```
127.0.0.1:5000/validate
Pretty print
{
  "message": "Blockchain is not valid.",
  "valid": false
}
```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.